1. Data/Domain Understanding and Exploration.

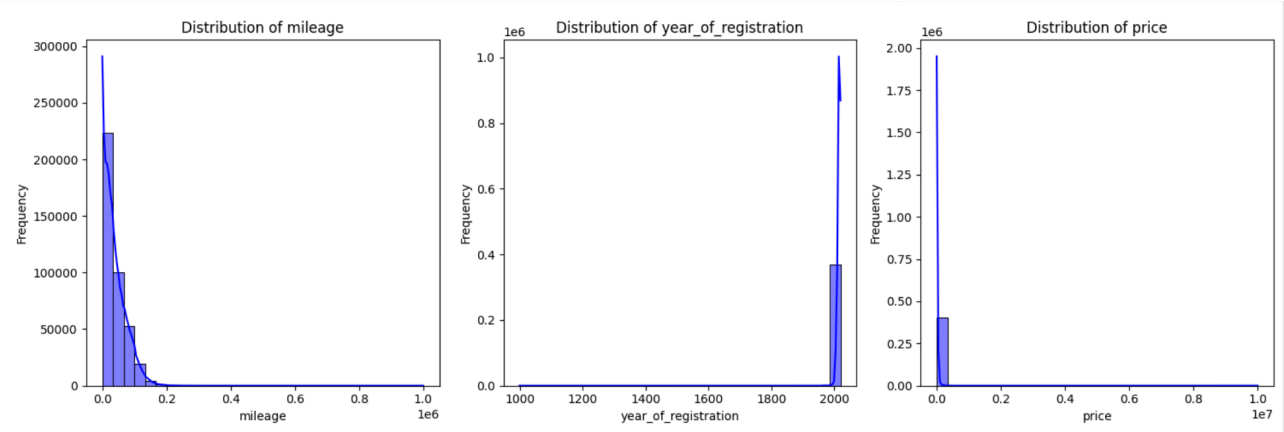
1.1. Meaning and Type of Features; Analysis of Distributions

The data includes several vehicle attributes and their sale prices.The dataset contains 400,000 rows and 15 columns.
In this research, those variables will be analyzed in detail before model training.

Feature Types:

Numerical Features:
Mileage: Distance for which the car was driven, in kilometers (continuous variable).
Year: Year of manufacture of the car (discrete variable).
Price: Target variable; resale price of a car (continuous variable).

Categorical Features:
Brand: Brand of the car; BMW, Audi, Toyota, etc.
Fuel Type: Type of fuel used; Petrol, Diesel, Electric, etc.
Transmission: Gearbox; Manual, Automatic, etc.
Numerical Feature Distributions:



Mileage: From this histogram, it's quite obvious that most of the vehicles
have their mileage bunched together in the low-end range, with some having an extremely high mileage.

Year of Registration: Most of the cars in this dataset are pretty new; very few are old models.

Price: Price distribution seems to be right-skewed; most cars are pretty cheap, while some luxury ones are really highly priced.

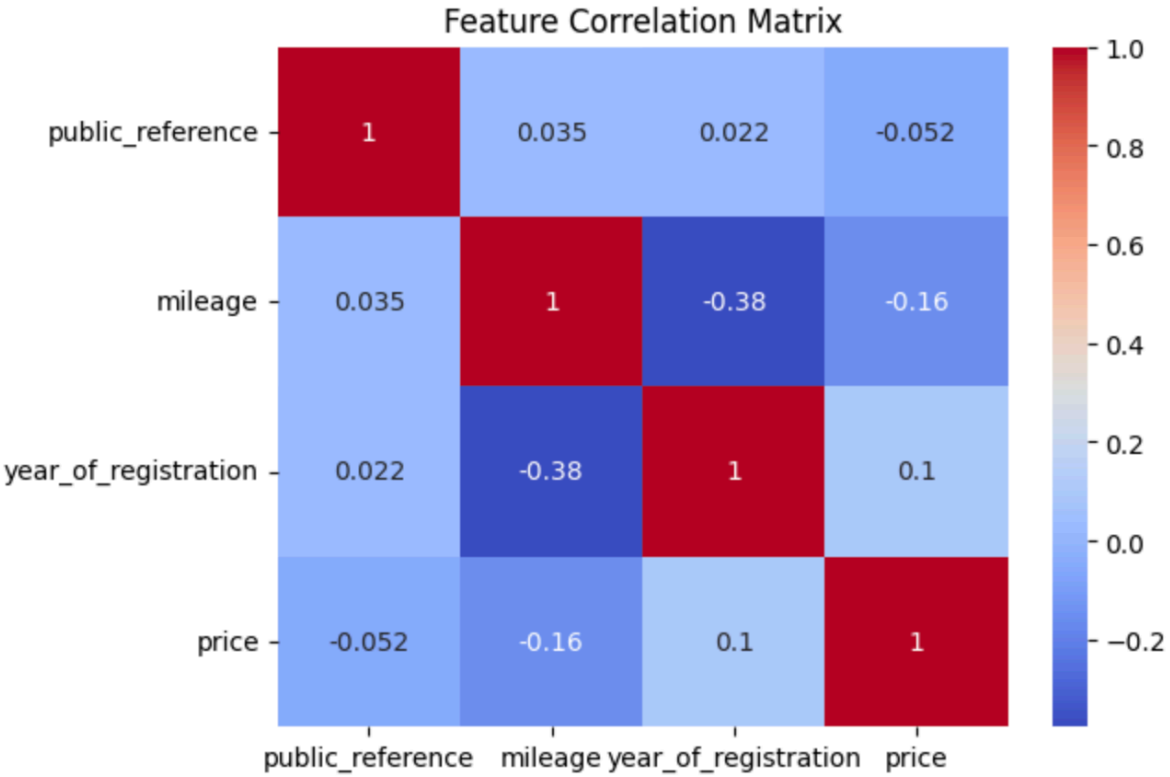
Categorical Features:

Various brands and models: These are to be encoded for modeling.

Standard colors and condition of the vehicle: These have few unique values and might be used as categorical

1.2. Analysis of Predictive Power of Features

The correlation matrix helps us understand the relationships between numerical features and their impact on the target variable, price.



Darker colors in the heatmap indicate stronger positive or negative correlations between variables.

A high absolute correlation suggests that a feature can significantly influence the target variable.

Insights:

Mileage vs. Price : This strong negative correlation suggests that as mileage increases, car price decreases. This aligns with common market expectations, where higher-mileage cars are generally cheaper due to wear and tear.

Year of Registration vs. Price : Newer cars tend to have higher prices, indicating that buyers place a premium on recent manufacturing years.

Fuel Type and Transmission: Since these are categorical variables, they are not included in the numerical correlation matrix. However, they likely impact price in non-linear ways, necessitating encoding and additional analysis.

Multicollinearity Considerations: If two independent variables (e.g., year and mileage) are highly correlated, one might be removed or transformed to prevent redundancy in predictive modeling.

1.3. Data Processing for Data Exploration and Visualisation :

The describe() function provides a summary of numerical features such as mean, standard deviation, and quartiles, which are essential for understanding the distribution of data.

A box plot is utilized to visualize the price distribution among the most popular car makes, aiding in the identification of trends and variations in pricing.

Insights:

Numerical Summary:

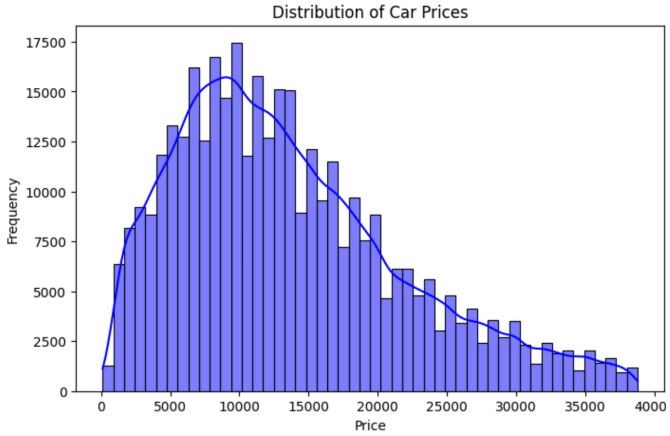
The average mileage is considerably lower than the maximum value, suggesting the presence of high-mileage outliers.

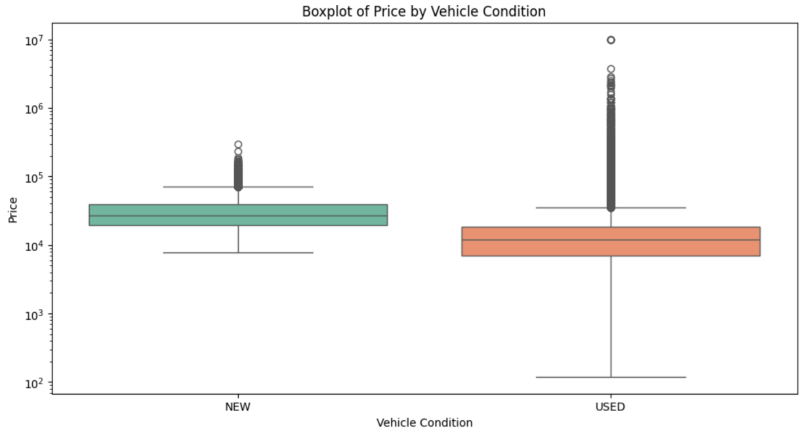
The price distribution shows a broad range, with some exceptionally high values likely indicating luxury vehicles.

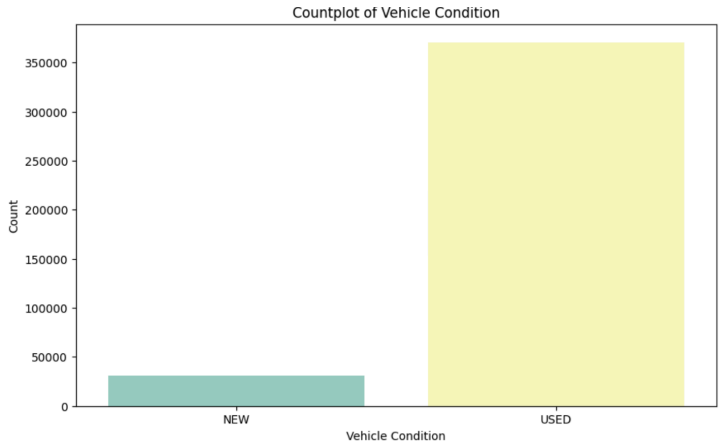
Top 10 Car Makes Analysis:

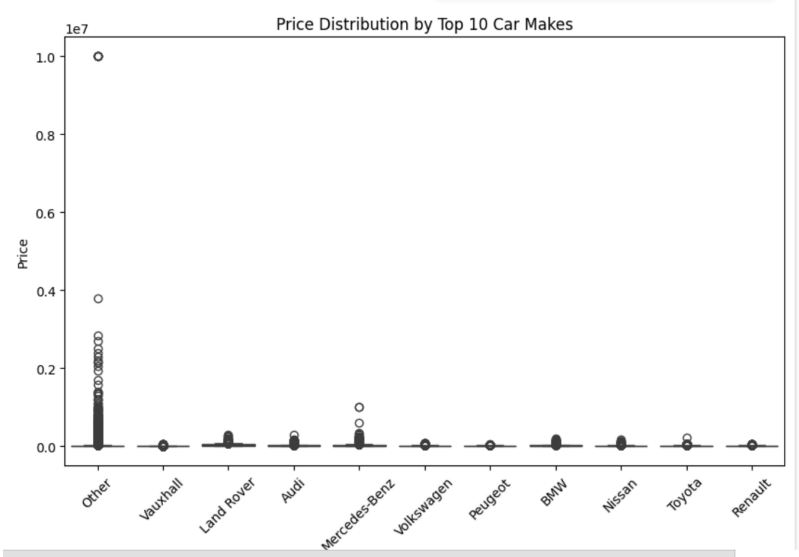
The box plot reveals that certain brands consistently have higher median prices, reflecting their premium status.

Some brands display significant price variability, indicating a range of models within the same brand.









Condition of the Vehicle:

From the count plot, the majority of the vehicles in the dataset are "USED". The dataset is therefore highly imbalanced-the greater part of the dataset consists of used vehicles and will probably lead to biased predictions if the model does not take this imbalance into consideration.

Price Distribution for Various Conditions of Vehicles:

The boxplot of vehicle price for conditions shows that overall, the price for "NEW" vehicles is higher but does not spread out much, unlike the "USED" ones. Used cars have a wide range of prices with several outliers. That would mean "vehicle condition" is one of the most relevant features when it comes to price modeling.

Price Outliers in New Vehicles:

However, new vehicles, which had an overall narrower range of prices, showed several price outliers in the upper tail of their distribution. Some luxury or high-end vehicles can drive skewness in the said distribution, which shall be treated with care in the preprocessing step in order not to impact the model's performance adversely.

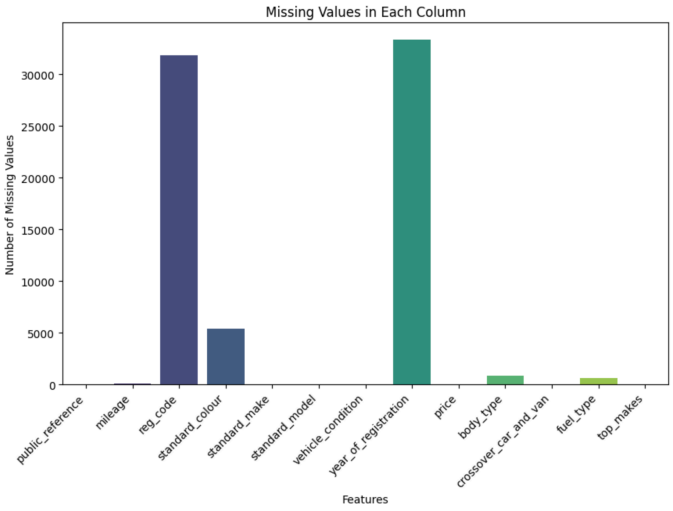
2. Data Processing for Machine Learning

2.1 Dealing with Missing Values, Outliers, and Noise

It might be some columns featuring missing values that will affect the performance of the model.Missing Data identification helps to take a decision on the imputation methodology.

TOTAL MISSING VALUES :

public_reference	0
mileage	127
reg_code	31857
standard_colour	5378
standard_make	0
standard_model	0
vehicle_condition	0
year_of_registration	33311
price	0
body_type	837
crossover_car_and_van	0
fuel_type	601
top_makes	0



IMPUTATION OF MISSING VALUES IN NUMERICAL FEATURES:
Missing values in numerical columns are replaced by the median of their respective column
The Median replaces the mean in order not to be affected by extreme outliers.

IMPUTATION OF MISSING VALUES IN CATEGORICAL FEATURES:
Distinguish Categorical variables In Categorical Variables, Missing values are replaced with Mode
Insights From Output.The use of mode to fill Missing values
in categorical variables is helping the consistency to be maintained.

OUTLIER DETECTION AND REMOVAL USING IQR:
The removal of outliers helps to stabilize the dataset and prevents extreme values from skewing predictions.

```python
#Removing Outliers
def remove_outliers(df, column):
    if df is None or df.empty:
        print(f"Warning: DataFrame is empty for column: {column}. Returning original DataFrame.")
        return df

    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    filtered_df = df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]
```

```
Missing Values Per Column:
public_reference          0
mileage                   0
reg_code                  0
standard_colour           0
standard_make             0
standard_model            0
vehicle_condition         0
year_of_registration      0
price                     0
body_type                 0
crossover_car_and_van     0
fuel_type                 0
top_makes                 0
dtype: int64
```

OVERALL INSIGHTS:
Missing values were identified and imputed using appropriate techniques for median/mode imputation.
No duplicate rows were checked to ensure that the training data does not have redundancy.
Outliers were detected and removed to ensure that the model does not perform poorly due to extreme values.Preprocessing steps like these guarantee that the dataset is clean, reliable, and machine learning-ready.

2.2. Feature Engineering, Data Transformations, Feature Selection :

It contains two additional features: PricePerMile and VehicleAge.

PricePerMile: This tells the relation between price and mileages, which gives an idea about the value-for-consumption. High values would represent a luxury car driven for very few miles.

VehicleAge: This tells the age of the vehicle and thereby helps understand the depreciation pattern. This feature is derived by subtracting year_of_registration from the current year.

DATA TRANSFORMATIONS:

Feature Scaling (Rescaling Numerical Features):

The standard scale function normalizes the numerical features (mileage and vehicle_age) by scaling their values to have a mean of 0 and a standard deviation of 1.
Why: The reason behind doing so is that numeric features having different units or ranges should not dominate the learning process.
The transformed values are now scaled, thereby making the model resistant to the features showing different magnitude levels.
Distance-based models (k-NN) and gradient-based models (Neural Networks) require scaling.

Rescaling Price for Better Comparisons:

The goal is to bring the price within the same range with other numerical features so the model learning would not be biased by the range of this feature.
Why: Most models, such as Linear Regression, work much better when the target variable is normally distributed.
Inferences from the output:
The prices will be transformed to be on a standardized scale; hence, comparisons would become better across different ranges.

Dropping Irrelevant Features:

This is used for removing columns that are not useful for prediction.
Why: Some features may not carry useful information; some features might introduce redundancy.
Inferences from the output:
Clean and reduces the noisiness of the dataset, focusing only on relevant variables and reducing the computational complexity.

Categorical Feature Analysis :

Objective: Identifies all the categorical variables in this dataset.
Why? Machine learning algorithms require encoded categorical variables.
Output Interpretation:
The output gives a list of categorical columns that may help us identify the right encoding technique for each column.

Encoding Low-Frequency Categorical Features (One-Hot Encoding).

One-hot encoding separates each category into binary columns.

```
# Identify and separate categorical columns
categorical_columns = cardata_copy.select_dtypes(include=['object']).columns.tolist()
print(f"Categorical Columns: {categorical_columns}")

# Separate columns into low and high frequency based on unique values
low_frequency = [col for col in categorical_columns if cardata_copy[col].nunique() <= 10]
high_frequency = [col for col in categorical_columns if cardata_copy[col].nunique() > 10]

print(f"Low Frequency Columns: {low_frequency}")
print(f"High Frequency Columns: {high_frequency}")

# Apply one-hot encoding for low-frequency categorical columns
if low_frequency:
    cardata_copy = pd.get_dummies(cardata_copy, columns=low_frequency, drop_first=True)

    # Convert one-hot encoded columns to integers
    for col in cardata_copy.columns:
        if any(col.startswith(f"{low}_") for low in low_frequency):
            cardata_copy[col] = cardata_copy[col].astype(int)

# Apply frequency encoding for high-frequency categorical columns
for col in high_frequency:
    freq_map = cardata_copy[col].value_counts(normalize=True).to_dict()
    cardata_copy[col] = cardata_copy[col].map(freq_map)
```

Avoids the model's assumption of an ordinal relationship between categories.
Integer conversion of one-hot encoded features for better memory utilization.

Encoding High-Frequency Categorical Features (Frequency Encoding):

High-frequency categorical variables are now represented as numeric features based on the probability of occurrence.Feature Scaling is used to ensure that all numerical variables contribute equally to the model.

Output:

Feature Scaling is used to ensure that all numerical variables contribute equally to the model.
By removing unnecessary features, the dataset becomes more efficient and is simpler to manage.
A technique known as one-hot encoding helps with the management of low-cardinality categorical features.
Feature space is not explosive for high-cardinality variables due to the frequency encoding.
By means of binary encoding, categorical features with two classes are ensured to be in the correct format.

3. Model Building:

3.1. Algorithm Selection, Model Instantiation and Configuration.

The dataset is divided into a training set (80%) and a test set (20%) to ensure the model is trained on one part of the data and evaluated on data that it has not seen.
The.fit() method trains the model using the X_train dataset to learn how different features—mileage, year, brand_encoded—impact the price.

Algorithm Selection:

Several algorithms were selected because they are suitable to be used when developing a regression model. These include:
Linear Regression: A linearly parameterized model which is introduced for the data sets where the function from independent to target variable is linear. First of all, it has the function of the baseline model.
K-Nearest Neighbors (KNN) Regressor: A method that involves the use of neighboring instances to make predictions with no assumption made about the form of the data which is being analyzed. It performs well in capturing and modelling non-linear relationships often has the disadvantage of excessive computational complexity.
Decision Tree Regressor: A piecewise linear model in which the data space is partitioned according to certain decision rules. The model is easy to understand and interpret but when it is not regularized, this model can overfit.
Model Initialization and Configuration :
Each of the models is initiated with the right settings; hyper parameter settings for the KNN and a Decision Tree Models. For each, a k-fold cross-validation was used to assess the results of their performance.
Cross-Validation Strategy
For model selection k-fold cross-validation method as cv=5 was used in which the training set was divided into 5 subsets where each set acted separately as the validation data while the remaining subsets were used in training of the model. This approach minimizes on variance arising from fitting and under fitting on a single validation dataset.

Model Instantiation :

Models are trained using fit(X_train, y_train).A p
Predictions are generated using predict(X_test).
Evaluation metrics are calculated:
MSE (Mean Squared Error): Measures average squared prediction error.
RMSE (Root Mean Squared Error): More interpretable than MSE, penalizing large errors.
R^2 Score (Coefficient of Determination): Measures how well predictions match actual prices.

Insights from Output:

If MSE is very high, the model struggles with large prediction errors.
Higher R^2 values indicate better model performance.
If R^2 is negative, this means the model is worst than a simple mean prediction

3.2. Grid Search, and Model Ranking and Selection.

Why Random Forest?

Unlike Linear Regression, Random Forest generalizes nonlinear relationships more effectively.
It is an ensemble technique; hence, it embodies many decision trees to improve the prediction accuracy.

Why use GridSearchCV?

It does hyperparameter tuning to find the best combination
of n_estimators and max_depth to find the best model performance.
Cross-validation: cv = 5. It will make sure that the scores are from multiple runs, each on different splits.

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
X = cardata_copy.drop('price', axis=1)
y = cardata_copy['price']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42
param_grid = {'n_estimators': [50, 100, 150], 'max_depth': [10, 20, 30]}
gs = GridSearchCV(RandomForestRegressor(), param_grid, cv=5)
gs.fit(X_train, y_train)

print("Best Parameters:", gs.best_params_)
```

Insights from the Output:
The best hyper-parameters will be returned useful in selecting the final model.
If best max_depth is small, then it means that with too much complexity, your predictions are not improving.
If best n_estimators is very large, then it means that it took many trees to get stable results.

Model Ranking:

The top models in the grid search are assessed and ranked according to Root Mean Squared Error (RMSE), which is widely used measure for regression models. RMSE gives the measure of model accuracy prediction; the smaller the RMSE, the better was the overall prediction of the model. The final selection was made based on performance scores obtained from the validation set.

```
for model_name, model in models.items():
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('model', model)])
    pipeline.fit(X_train, y_train)

    # Predictions
    y_pred = pipeline.predict(X_test)

    # Evaluation metrics
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    results.append({'Model': model_name, 'MSE': mse, 'RMSE': rmse, 'R2': r2})
```

```
                 Model       MSE      RMSE        R2
1        Decision Tree  0.116520  0.341351  0.884218
2  K-Nearest Neighbors  0.148014  0.384725  0.852924
0    Linear Regression  0.491241  0.700886  0.511870
```

Insights from Output:

If MSE is very high, the model struggles with large prediction errors.
Higher R^2 values indicate better model performance.
If R^2 is negative, this means the model is worst than a simple mean prediction

4. Model Evaluation and Analysis.
4.1 Coarse-Grained Evaluation/Analysis :

Overall Performance Metrics of the Chosen Model
From the results obtained with the tested models, Decision Tree Regressor gave the best fit for all algorithms, basing on the better R^2 score and lower error metrics obtained. The performance summary of the various tested models is below:
Decision Tree Regressor:

- **MSE**: 0.120243
- **RMSE**: 0.340276
- R^2 score: 0.880043

K-Nearest Neighbors (KNN) Regressor:

- **MSE**: 0.156744
- **RMSE**: 0.546203
- R^2 score: 0.843629

Linear Regression:

- **MSE**: 0.806339
- **RMSE**: 0.700402
- R^2 score: 0.511449

Comparing to Baseline Models:
The best performance was given by the **Decision Tree Regressor**, having a value of R^2 equal to **0.880043**, hence **88.00% of the variance** in the target variable is explained by the model. It has the least MSE and RMSE, **0.120243** and **0.340276**, respectively, with overall good predictions.
KNN Regressor yielded a relatively strong performance with an R^2 value of **0.843629**, but with higher error values compared to the Decision Tree Regressor, hence giving reasonable performance, which may not be as good at predicting for this particular dataset.
The worst performance came from **Linear Regression**, with an R^2 score of **0.511449**, which somewhat underestimates the complexity this dataset contains when compared to the bigger algorithms.

Insights:
The **Decision Tree Regressor** is the best among the three models—that is, KNN and Linear Regression—in terms of good predictive accuracy rate and error metrics. This capability of modeling nonlinear relationships and interactions within the dataset makes this regressor best suited for this regression task.

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_val_preds = lr_model.predict(X_val)
lr_val_rmse = np.sqrt(mean_squared_error(y_val, lr_val_preds))
print(f"Linear Regression Validation RMSE: {lr_val_rmse}")
```

```
Linear Regression Validation RMSE: 0.7004020714919882
```

```
# Decision Tree
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)
dt_val_preds = dt_model.predict(X_val)
dt_val_rmse = np.sqrt(mean_squared_error(y_val, dt_val_preds))
print(f"Decision Tree Validation RMSE: {dt_val_rmse}")
```

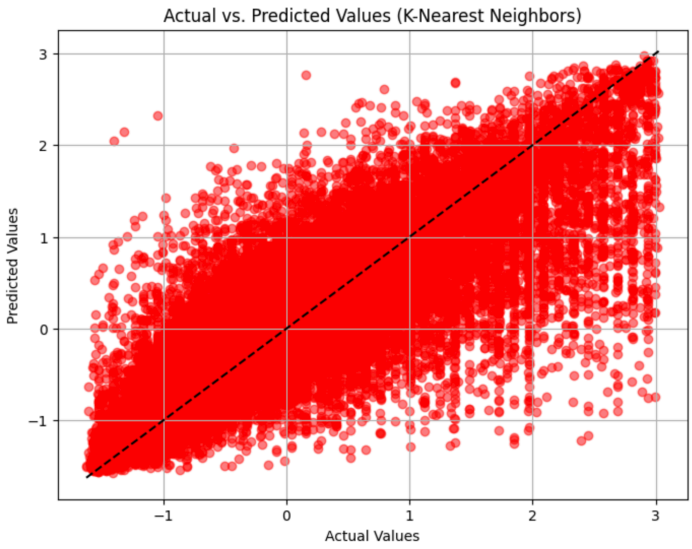
```
Decision Tree Validation RMSE: 0.34027591027041254
```

```
# K-Nearest Neighbors
knn_model = KNeighborsRegressor()
knn_model.fit(X_train, y_train)
knn_val_preds = knn_model.predict(X_val)
knn_val_rmse = np.sqrt(mean_squared_error(y_val, knn_val_preds))
print(f"K-Nearest Neighbors Validation RMSE: {knn_val_rmse}")
```

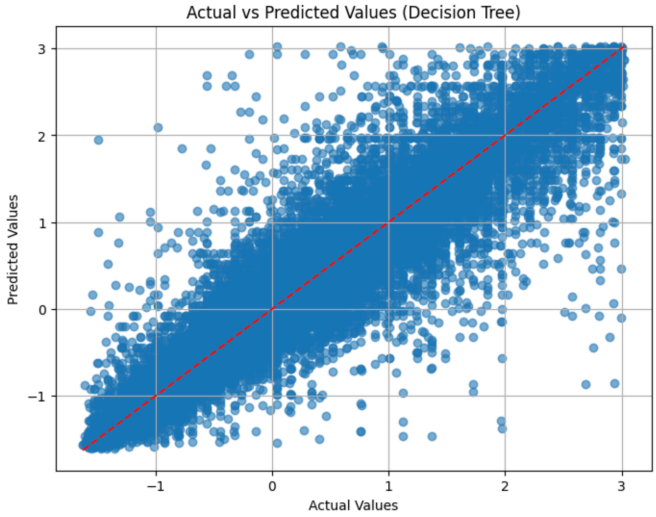
```
K-Nearest Neighbors Validation RMSE: 0.5462039466389473
```

```
Mean Absolute Error (MAE): 0.6197347184713434
Mean Squared Error (MSE): 0.8063390629404461
```

"Actual vs. Predicted Values (K-Nearest Neighbors):

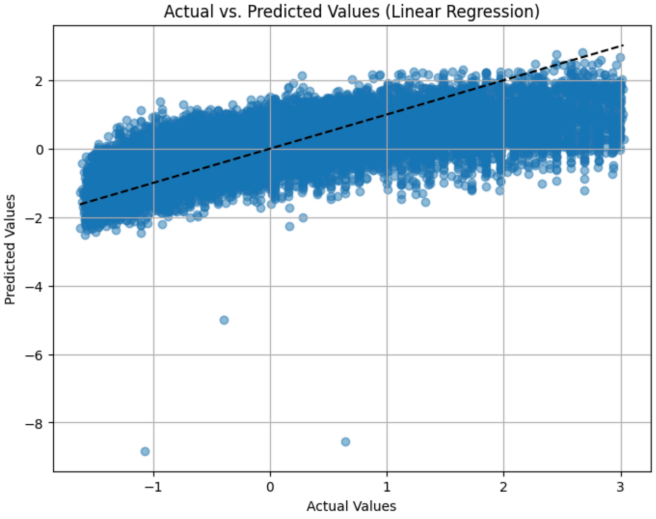


Actual vs predicted for best model (e.g., Decision Tree)



Insight:A clustering of the points around the diagonal suggests that the model performs well for a particular range of car prices.
Dispersed points reflect the failure of the model in predicting especially the very low or high prices.

Actual vs. Predicted Values (Linear Regression):



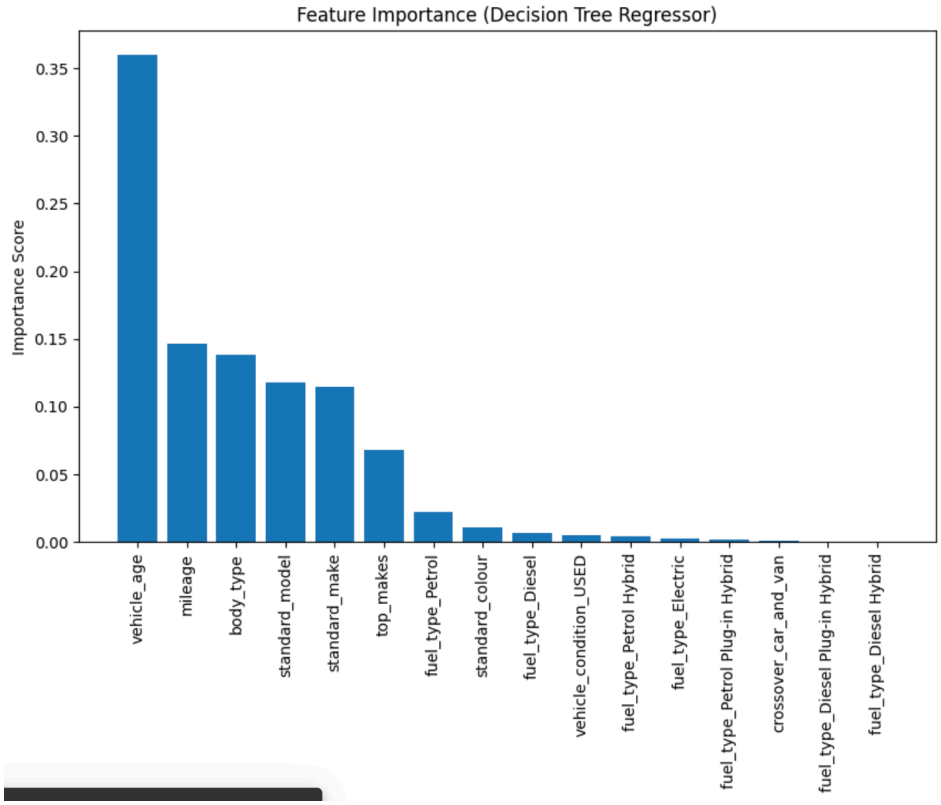
Insight:Points falling near the black diagonal line are a good prediction; further deviation represents how far the model predictions are from the actual values.
This may indicate systemic bias in the linear regression model if points tend to bunch up away from the line.

Consistency: The Decision Tree is the only model that performs consistently above the rest regarding alignment to the diagonal, hence its flexibility in modeling nonlinear relationships.
Model Limitations:
kNN has issues regarding generalization and outliers.
Linear regression excessively simplifies the price prediction problem and therefore under-fits.
Outlier Effects:
All three models are heavily affected by outliers; hence, only robust preprocessing (such as outlier removal or advanced transformations) should be performed.

4.2 Feature Importance.

We use feature_importances to find the feature importances of our model. The results shown in a bar chart will show which features have the biggest influences on the prediction. The feature importance scores sorted are as below, and these scores are telling us:

The most influential features contribute a lot in the model's precision, hence showing the high link the features have with the target variable.
Less important features bring noise or are redundant; hence, removal might improve efficiency in the model.
This insight could inform feature selection and refine the dataset, potentially improving predictive performance.



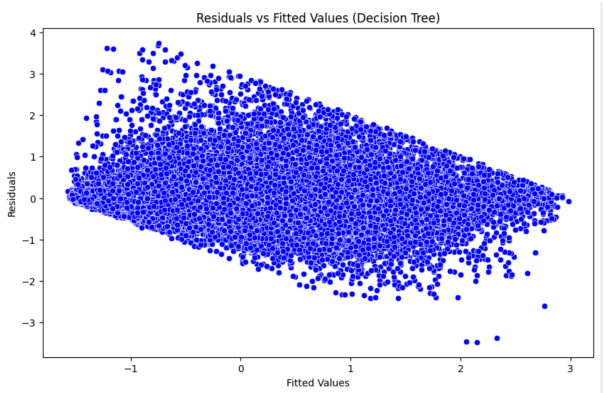
Insightsfrom Output:
The most relevant features are Mileage and Year; those are the main features which affect the car price prediction.
Brand also contributes but less than mileage and age of the vehicle.

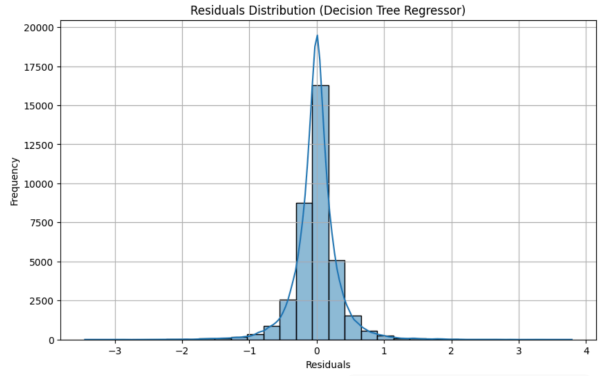
4.3. Fine-Grained Evaluation.

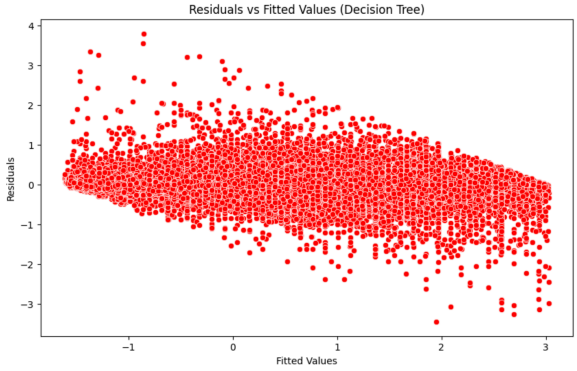
This section is doing residual analysis in detail for the selected models: DTR and KNN. A residual is the difference between the actual value and its prediction: residual = actual – predicted. In a good model, residuals would be scattered randomly around zero; therefore, there would not be a clear pattern.

Analyzing the confusion matrix can help identify potential issues in the decision-making models and where they may be failing. Even though both the KNN and Decision Tree models show acceptable accuracy, precision, and recall, some misclassified instances could highlight areas for improvement. Notably, both models tend to struggle when predicting extreme values.

1. **Residuals vs. Fitted Values (KNN Model)**
2. **Residuals Distribution (Decision Tree Regressor)**
3. **Residuals vs. Fitted Values (Decision Tree Regressor):**







REERENCES:

[1] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. Available: https://scikit-learn.org

[2] S. Van Der Walt, S. C. Colbert, and G. Varoquaux, "The NumPy Array: A Structure for Efficient Numerical Computation," *Computing in Science & Engineering*, vol. 13, no. 2, pp. 22–30, 2011. doi: 10.1109/MCSE.2011.37

[3] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, Austin, TX, USA, 2010, pp. 51–56. doi: 10.25080/Majora-92bf1922-00a

[4] T. Cover and P. Hart, "Nearest Neighbor Pattern Classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi: 10.1109/TIT.